

Fiche IoT

Réf. Développement – Langage Arduino

Présentation



Arduino est la marque d'une plateforme de prototypage open-source qui permet de créer des objets électroniques interactifs, IoT, à partir de cartes électroniques matériellement libres sur lesquelles se trouve un microcontrôleur.

Le langage Arduino repose entièrement sur le langage C/C++ dont il exploite toutes les possibilités en y intégrant la gestion des broches E/S (moyens de communication) présentes sur les diverses cartes électroniques (IoT) compatibles.

Le langage Arduino peut être divisé en trois parties principales :

- Partie 1 - **Les Fonctions**,
- Partie 2 – **Les Données** (Variables et Constantes),
- Partie 3 – **Les Structures**.

Partie 1- Les Fonctions

Les Fonctions Standards du langage

E/S Numériques

digitalRead()	<code>digitalRead(pin)</code>
digitalWrite()	<code>digitalWrite(pin, value)</code>
pinMode()	<code>pinMode(pin, mode)</code>

E/S Analogique

analogRead()	<code>analogRead(pin)</code>
analogReference()	<code>analogReference(type)</code>
analogWrite()	<code>analogWrite(pin, value)</code>

E/S Spéciales

noTone()	<code>noTone(pin)</code>
pulseIn()	<code>pulseIn(pin, value [,timeout])</code>
pulseInLong()	<code>pulseInLong(pin, value [,timeout])</code>
shiftIn()	<code>shiftIn(dataPin, clockPin, bitOrder)</code>
shiftOut()	<code>shiftOut(dataPin, clockPin, bitOrder, value)</code>
tone()	<code>tone(pin, frequency [,duration])</code>

Temps

delay()	<code>delay(ms)</code>
delayMicroseconds()	<code>delayMicroseconds(us)</code>
micros()	<code>micros()</code>
millis()	<code>millis()</code>

Mathématiques

abs()	<code>abs(x)</code>
constrain()	<code>constrain(x, a, b)</code>
map()	<code>map(value, fromLow, fromHigh, toLow, toHigh)</code>
max()	<code>max(x, y)</code>
min()	<code>min(x, y)</code>
pow()	<code>pow(base, exponent)</code>
sq()	<code>sq(x)</code>
sqrt()	<code>sqrt(x)</code>

Trigonométrie

cos()	<code>cos(rad)</code>
sin()	<code>sin(rad)</code>
tan()	<code>tan(rad)</code>

Nombres Aléatoires

random()	<code>random([min,] max)</code>
randomSeed()	<code>randomSeed(seed)</code>

Caractères

isAlpha()	isAlpha (thisChar)
isAlphaNumeric()	isAlphaNumeric (thisChar)
isAscii()	isAscii (thisChar)
isControl()	isControl (thisChar)
isDigit()	isDigit (thisChar)
isGraph()	isGraph (thisChar)
isHexadecimalDigit()	isHexadecimalDigit (thisChar)
isLowerCase()	isLowerCase (thisChar)
isPrintable()	isPrintable (thisChar)
isPunct()	isPunct (thisChar)
isSpace()	isSpace (thisChar)
isUpperCase()	isUpperCase (thisChar)
isWhitespace()	isWhitespace (thisChar)

Bits et Octets

bit()	bit (n)
bitClear()	bitClear (x, n)
bitRead()	bitRead (x, n)
bitSet()	bitSet (x, n)
bitWrite()	bitWrite (x, n, b)
highByte()	highByte (x)
lowByte()	lowByte (x)

Communication ([Stream](#))

Serial	if(Serial) available() availableForWrite() begin() end() find() findUntil() flush() parseFloat() parseInt() peek() print() println() read() readBytes() readBytesUntil() readString() readStringUntil() setTimeout() write() serialEvent()
Wire	#include <Wire.h> begin() end() requestFrom() beginTransmission() endTransmission() write() available() read() setClock() onReceive() onRequest() setWireTimeout() clearWireTimeoutFlag() getWireTimeoutFlag()
Ethernet	Bibliothèque spécialisée dans la gestion des communications réseau.
SD	Bibliothèque spécialisée dans la gestion de la carte SD.

USB

Keyboard	Keyboard.begin() Keyboard.end() Keyboard.press() Keyboard.print() Keyboard.println() Keyboard.release() Keyboard.releaseAll() Keyboard.write()
Mouse	Mouse.begin() Mouse.click() Mouse.end() Mouse.move() Mouse.press() Mouse.release() Mouse.isPressed()

Interruptions Externes

attachInterrupt()	<code>attachInterrupt (digitalPinToInterrupt (pin) , ISR, mode)</code>
detachInterrupt()	<code>detachInterrupt (digitalPinToInterrupt (pin))</code>

Interruptions (Activation/Désactivation)

interrupts()	<code>interrupts ()</code>
noInterrupts()	<code>noInterrupts ()</code>

Conversion de Type

(unsigned int)	(unsigned int)x
(unsigned long)	(unsigned long)x
byte()	byte(x) (byte)x
char()	char(x) (char)x
float()	float(x) (float)x
int()	int(x) (int)x
long()	long(x) (long)x
word()	word(x) word(h, l) (word)x
String()	String(val) String(val, base) String(val, decimalPlaces)

Les Fonctions Personnelles

On vient de la voir, le langage Arduino fournit un certain nombre de fonctions standards. Cependant, vous serez amené à devoir créer vos propres fonctions pour réaliser des traitements particuliers. Le cas typique de la création de fonctions est quand il faut effectuer plusieurs fois la même action à plusieurs endroits dans le programme. On appelle également cela du *refactoring* de code. Cela permet d'éviter une multitude de lignes de codes inutiles qui se répètent sans cesse. Dans une fonction, on définit le code à exécuter. On aura plus qu'à faire appel à cette fonction n'importe où dans le programme.

La normalisation des fragments de code en fonctions présente plusieurs avantages :

- Les fonctions aident le programmeur à rester organisé. Souvent, cela aide à conceptualiser le programme.
- Les fonctions codifient une action en un seul endroit, de sorte que la fonction doit seulement être pensée et déboguée une fois.
- Cela réduit également les risques d'erreur de modification, si le code doit être modifié.
- Les fonctions réduisent considérablement le nombre de lignes de code dans un fichier de programme (ou esquisse), car ces sections de codes sont réutilisées plusieurs fois.
- Les fonctions facilitent la réutilisation du code dans d'autres programmes en le rendant modulaire, et l'utilisation de fonctions rend souvent le code plus lisible.

Il y a deux fonctions obligatoires sur Arduino. La fonction d'initialisation « **setup()** » et la fonction du programme principal « **loop()** ». Les autres fonctions doivent être créées en dehors des crochets de ces deux fonctions.

Définition

La syntaxe la plus courante pour définir une fonction est la suivante :

Type de retour	Nom_de_la_fonction	(Argument1, Argument 2, ...)
{	Code ; // traitement(s) quelconque(s)	
}		

- **Type de retour** : Il s'agit du type de donnée retournée par la fonction. Mettre « void » pour une fonction qui ne renvoie rien.
- **Nom_de_la_fonction** : Il s'agit de l'identifiant par lequel la fonction va être appelée. Ce nom n'accepte pas le caractère « espace » ou « - » (*tiret du 6*).
- **Argument(s)** : Il s'agit du/des paramètres à passer à la fonction. Ces derniers peuvent être de n'importe quel type.
- **Code** : Corp de la fonction contenant toutes les lignes de traitements à exécuter.

Déclaration

Une fonction est déclarée en dehors de toute autre fonction, au-dessus de la fonction **setup()** ou en dessous de la fonction **loop()**. Nous pouvons déclarer la fonction de deux manières différentes :

1. **La première façon** consiste simplement à écrire la partie de la fonction appelée implémentation de fonction au-dessus de la fonction **setup()**, avec :
 - Type de retour de fonction
 - Nom de la fonction
 - Type et nom d'argument de fonction
 - Le corps de la fonction (instructions à l'intérieur de la fonction en cours d'exécution lorsque la fonction est appelée)

Exemple

```
int sum_func(int x, int y) {    // Declaration de fonction
  int z = 0;
  z = x+y ;
  return z;                    // Retour du resultat
}

void setup() {
  ...                          // Initialisation(s) quelconque(s)
}

void loop() {
  int result = 0 ;
  result = sum_func(5,6) ;    // Appel de la fonction
}
```

2. **Une autre manière** de déclarer une fonction, consiste à déclarer en premier lieu un **prototype** de fonction au-dessus de la fonction **setup()**. Ensuite, ce prototype fera appel à son **implémentation**, située en-dessous de la fonction **loop()**, avec :

- Type de retour de fonction
- Nom de la fonction
- Type et nom d'argument de fonction
- Le corps de la fonction (instructions à l'intérieur de la fonction en cours d'exécution lorsque la fonction est appelée)

Le prototype de fonction n'a besoin que du type d'argument et doit être suivi d'un « ; » (point-virgule).

Exemple

```
int sum_func(int , int ) ; // Prototype de la fonction (le nom d'argument n'est pas nécessaire)

void setup() {
    ... // Initialisation(s) quelconque(s)
}

void loop() {
    int result = 0 ;
    result = sum_func(5,6) ; // Appel de la fonction
}

int sum_func(int x, int y) { // Déclaration de la fonction
    int z = 0;
    z = x+y ;
    return z; // Retour du resultat
}
```

Pour des raisons de lecture et de normalisation, on préférera cette seconde manière de déclarer nos fonctions qui sera plus proche de la répartition prototype/implémentation de nos fichiers « .h » et « .cpp » que l'on verra un peu plus loin → [Les Extensions de Fichiers « .cpp » et « .h »](#).

Partie 2 – Les Données (Variables et Constantes)

Une donnée **variable** est une entité symbolique dont la **valeur** est susceptible d’être modifiée en cours d’exécution du programme. A l’inverse, une donnée **constante** voit sa valeur fixée par programmation et toute modification de cette dernière au cours de l’exécution du programme entraine un « **Fatal Error** » et l’arrêt du programme.

Dans certain langage comme le C/C++, l’obligation de typer une variable, c’est-à-dire lui associer un type de donnée qu’elle peut stocker, implique une bonne connaissance des valeurs, limites et occupation mémoire, de chacun de ces types.

La bonne compréhension et maitrise de cette notion permet d’optimiser l’utilisation de la mémoire de nos IoT à ce qui est strictement nécessaire et pas plus.

Les Données – Constantes

Le langage Arduino, outre la possibilité de définir des constantes personnalisées, propose quelques constantes par défaut telles que :

HIGH ('0x1') LOW ('0x0')	byte
INPUT ('0x00') OUTPUT ('0x01') INPUT_PULLUP ('0x02') INPUT_PULLDOWN_16 ('0x04') valeur spécifique ESP8266	byte
LED_BUILTIN ('2') valeur spécifique ESP8266 correspondant à la pin D4	byte
true ('1') false ('0')	bool

Il existe 3 moyens différents de déclarer des données constantes :

- Avec le mot-clé « **const** ». C’est la manière la plus recommandée pour déclarer une constante dans un programme.
- Avec la directive « **#define** ». La constante sera ici prise en charge par le préprocesseur.
- Avec le mot-clé « **enum** ». Ici **enum** est très pratique pour déclarer des constantes de noms et non de valeurs.

- Déclaration avec le mot-clé « **const** »

Une déclaration de ce type se définit par le mot-clé « **const** », par le type de donnée, le nom de la constante et sa valeur. Il s'agit de la forme la plus robuste. Le compilateur connaît le type de la constante et bloquera cette donnée. Elle sera donc invariable, peu importe le comportement de votre programme. C'est la forme à utiliser en priorité.

Exemple

```
const float PI = 3.14 ;
```

- Déclaration avec la directive « **#define** »

En réalité « **#define** » ne fait pas partie du langage Arduino. C'est ce qu'on appelle une directive ou fonction préprocesseur ou encore macro. C'est une fonction qui va être appelée par le compilateur avant que ce dernier effectue son travail de compilation.

Exemple

```
#define PI 3.14
```

Comme on peut le voir dans l'exemple, le type de donnée ne doit pas être spécifié. Du fait que ce n'est pas du code C, la syntaxe n'accepte pas le point-virgule ou le signe « = ».

Avant la compilation, la directive « **#define** » va remplacer toutes les références 'PI' par la valeur '3.14'. La donnée n'est pas verrouillée, car elle n'existera pas dans le programme. Le programme ne verra que des '3.14' partout. Dans certains cas, cela peut poser des problèmes. À proscrire si possible (préférez « **const** » !).

- Déclaration avec le mot-clé « **enum** »

Ce type de déclaration est une arme puissante lorsqu'il s'agit de déclarer des constantes dont on ne veut pas forcément connaître la valeur. Pratique pour ne disposer que de noms au lieu de valeurs numériques ! On s'en sert, entre autres, pour exprimer un état. C'est souvent utilisé dans les protocoles de communication. Ici, on va prendre un exemple simple : l'état d'un feu tricolore.

Exemple

```
enum etatfeu_t { VERT, ORANGE, ROUGE } ;  
etatfeu_t mavariable = VERT ;
```

Ici, on a défini 3 constantes : VERT, ORANGE et ROUGE. Les valeurs n'ont pas d'importance, c'est le compilateur qui va assigner des valeurs aux noms et « réfléchir » avec ces valeurs. Généralement le compilateur va assigner des valeurs dans l'ordre croissant en commençant de 0. Ici VERT vaut 0, ORANGE vaut 1, ROUGE vaut 2.

Les Données - Variables

Pour déclarer une donnée variable dans un programme, rien de plus simple. Chaque variable est déclarée selon son type de donnée. Une variable peut être initialisée à la création ou non. On peut également déclarer plusieurs variables du même type en même temps.

Exemple

```
int variableA;           // Déclaration d'une variable de type « int » nommée variableA.
variableA = 0;           // Initialisation de la variableA avec la valeur 0.
int variableB, variableC; // Déclaration de 2 variables en même temps.
float variableD = 0.0    // Utilisation du point '.' pour définir le type « float »
```

Type de donnée		Type et signe	Limites	Taille mémoire (octets)
Valeurs binaires	bool	entier non signé	0 ou 1	1
Valeurs numérique entières <u>signées</u>	int short int16_t	entier signé	-32 768 à +32 767	2
	long	entier signé	- 2 147 483 648 à +2 147 483 647	4
Valeurs numérique entières <u>non signées</u>	byte uint8_t	entier non signé	0 à +255	1
	unsigned int uint16_t	entier non signé	0 à +65 535	2
	word uint16_t	entier non signé	0 à +65 535	2
	unsigned long ulong uint32_t	entier non signé	0 à +4 294 967 295	4
Valeurs numériques à virgules <u>signées</u>	float	décimal signé	-3,4028235 10 ³⁸ à +3,4028235 10 ³⁸	4
	double	décimal signé	-3,4028235 10 ³⁸ à +3,4028235 10 ³⁸	4
Caractères	char	entier signé (<i>code ASCII</i>)	0 à +127	1
	unsigned char	entier non signé (<i>code ASCII étendu</i>)	0 à +255	1
Tableau	array	tableau de ‘n’ données de type ‘x’	-	‘n’ * sizeof(‘x’)
Chaine	string	tableau de ‘n’+1 char	-	‘n’+1

size_t	« size_t » est un type particulier qui permet de représenter la taille, en octets, de n'importe quelle donnée. Souvent utilisé pour stocker le retour de sizeof()
------------------------	--

bool (boolean)

Un « **bool** » est un booléen. Il s'agit d'une donnée ne pouvant prendre que **2 valeurs** : **true** (*vrai*) ou **false** (*faux*). Il est également possible de remplacer **true** par **1** et **false** par **0** pour un booléen. Chaque donnée booléenne occupe un octet (byte) de mémoire.

Exemple

```
bool val = false;           // déclaration d'une variable booléenne et initialisation à false
bool state = true;         // déclaration d'une variable booléenne et initialisation à true
```

byte

Une donnée « **byte** » peut stocker **un nombre non signé sur 8 bits**, de 0 à 255.

Exemple

```
byte m = 42;                /* Déclaration d'une variable de type byte et initialisation de cette
                             variable à la valeur 42 */
```

int

Les données « **int** » représentent les données de stockage des **nombre entiers**. Il s'agit du type de donnée principal pour le stockage des nombres. **int** stocke une valeur sur 16 bits, soit 2 octets. Cela donne une fourchette de **-32 768 à 32 767** (soit une valeur minimale de -2^{15} et une valeur maximale de $2^{15}-1$).

Attention : la taille d'une donnée « int » peut varier d'une carte IoT à l'autre. Veillez à bien contrôler avant toute utilisation.

Exemple

```
int cpt = -12500;           // Déclaration d'une variable int et initialisation à la valeur -12500
```

unsigned int

Les « **unsigned int** » fonctionnent de la même manière que les « **int** ». La différence ici, c'est que les nombres négatifs ne sont plus pris en compte. Le nombre est toujours encodé sur 2 bytes, soit 16 bits, soit 2 octets. La plage positive est donc plus grande : **de 0 à 65 535** ($2^{16}-1$).

Exemple

```
unsigned int cpt = 42500;    // Déclaration d'une variable de type unsigned int à la valeur 42500
```

word

Un « **word** » peut stocker un nombre non signé sur 16 bits. Le fonctionnement d'une donnée « **word** » est donc exactement le même qu'un « **unsigned int** ».

Exemple

```
word cpt = 42500; // Déclaration d'une variable de type word à la valeur 42500
```

long

Les données « **long** » peuvent stocker des **nombre entiers** sur 4 bytes, soit 32 bits : **de -2 147 483 648 (-2³¹) à 2 147 483 647 (2³¹+1)**.

Exemple

```
long vitLumiere = 299792458; /* Déclaration d'une variable de type long à la valeur 299 792 458 */
```

unsigned long

Les données « **unsigned long** » peuvent stocker des **nombre non signés entiers** sur 32 bits : **de 0 à 4 294 967 295 (2³²-1)**.

Exemple

```
unsigned long max = 4294967295; /* Déclaration d'une variable de type unsigned long à la valeur 4 294 967 295 */
```

short

Une donnée de type « **short** » peut stocker **un nombre entier** sur 16 bits quel que soit la carte IoT utilisée : **de -32 768 à 32 767** (soit une valeur minimale de -2¹⁵ et une valeur maximale de 2¹⁵-1).

Exemple

```
short max = 32767; // Déclaration d'une variable de type short à la valeur 32 767
```

float

Le type de données « **float** » permet de stocker **un nombre à virgule flottante**. Les variables float sont souvent utilisées pour approcher les valeurs analogiques et continues car la donnée **float** possède une plus grande résolution que les nombres entiers.

La plage d'un nombre float est la plus grande : **de -3,4028235^{E+38} à 3,4028235^{E+38}**. Ce nombre est stocké sur 32 bits, soit 4 octets.

Exemple

```
float pi = 3.14159265359; /* Déclaration d'une variable de type float à la valeur 3.14159265359 */
```

double

Une donnée « **double** » fonctionne exactement de la même manière qu'une donnée « **float** ». Elle occupe également 32 bits, sans gain de précision supplémentaire.

Exemple

```
double pi = 3.14159265359; /* Déclaration d'une variable de type double à la valeur 3.14159265359 */
```

char

Les « **char** » sont des données pouvant contenir **un caractère**. C'est un type de donnée qui occupe un octet de mémoire. Un caractère s'écrit dans une variable **char** entre **guillemets simples** comme ceci : 'A'. Pour écrire plusieurs caractères, ce qu'on appelle aussi une chaîne de caractères (string), on utilisera les **guillemets doubles** comme ceci : "ABC".

Mais derrière chaque caractère se cache un nombre. Vous pouvez voir l'encodage spécifique dans le tableau des **caractères ASCII** ci-dessous. Cela signifie qu'il est possible d'effectuer des opérations arithmétiques sur des caractères, dans lesquels la valeur ASCII du caractère est utilisée. Par exemple, 'A' + 1 correspond à la valeur 66, car la valeur ASCII du caractère 'A' est 65.

DEC	OCT	HEX	BINARY	SYMBOL	DEC	OCT	HEX	BINARY	SYMBOL	DEC	OCT	HEX	BINARY	SYMBOL	DEC	OCT	HEX	BINARY	SYMBOL
0	0000	00	00000000	NULL	32	0040	20	00100000		64	0100	40	01000000	@	96	0140	60	01100000	`
1	0001	01	00000001	SOH	33	0041	21	00100001	!	65	0101	41	01000001	A	97	0141	61	01100001	a
2	0002	02	00000010	STX	34	0042	22	00100010	"	66	0102	42	01000010	B	98	0142	62	01100010	b
3	0003	03	00000011	ETX	35	0043	23	00100011	#	67	0103	43	01000011	C	99	0143	63	01100011	c
4	0004	04	00000100	EOT	36	0044	24	00100100	\$	68	0104	44	01000100	D	100	0144	64	01100100	d
5	0005	05	00000101	ENQ	37	0045	25	00100101	%	69	0105	45	01000101	E	101	0145	65	01100101	e
6	0006	06	00000110	ACK	38	0046	26	00100110	&	70	0106	46	01000110	F	102	0146	66	01100110	f
7	0007	07	00000111	BEL	39	0047	27	00100111	'	71	0107	47	01000111	G	103	0147	67	01100111	g
8	0010	08	00001000	BS	40	0050	28	00101000	(	72	0110	48	01001000	H	104	0150	68	01101000	h
9	0011	09	00001001	HT	41	0051	29	00101001	)	73	0111	49	01001001	I	105	0151	69	01101001	i
10	0012	0A	00001010	LF	42	0052	2A	00101010	*	74	0112	4A	01001010	J	106	0152	6A	01101010	j
11	0013	0B	00001011	VT	43	0053	2B	00101011	+	75	0113	4B	01001011	K	107	0153	6B	01101011	k
12	0014	0C	00001100	FF	44	0054	2C	00101100	,	76	0114	4C	01001100	L	108	0154	6C	01101100	l
13	0015	0D	00001101	CR	45	0055	2D	00101101	-	77	0115	4D	01001101	M	109	0155	6D	01101101	m
14	0016	0E	00001110	SO	46	0056	2E	00101110	.	78	0116	4E	01001110	N	110	0156	6E	01101110	n
15	0017	0F	00001111	SI	47	0057	2F	00101111	/	79	0117	4F	01001111	O	111	0157	6F	01101111	o
16	0020	10	00010000	DLE	48	0060	30	00110000	0	80	0120	50	01010000	P	112	0160	70	01110000	p
17	0021	11	00010001	DC1	49	0061	31	00110001	1	81	0121	51	01010001	Q	113	0161	71	01110001	q
18	0022	12	00010010	DC2	50	0062	32	00110010	2	82	0122	52	01010010	R	114	0162	72	01110010	r
19	0023	13	00010011	DC3	51	0063	33	00110011	3	83	0123	53	01010011	S	115	0163	73	01110011	s
20	0024	14	00010100	DC4	52	0064	34	00110100	4	84	0124	54	01010100	T	116	0164	74	01110100	t
21	0025	15	00010101	NAK	53	0065	35	00110101	5	85	0125	55	01010101	U	117	0165	75	01110101	u
22	0026	16	00010110	SYN	54	0066	36	00110110	6	86	0126	56	01010110	V	118	0166	76	01110110	v
23	0027	17	00010111	ETB	55	0067	37	00110111	7	87	0127	57	01010111	W	119	0167	77	01110111	w
24	0030	18	00011000	CAN	56	0070	38	00111000	8	88	0130	58	01011000	X	120	0170	78	01111000	x
25	0031	19	00011001	EM	57	0071	39	00111001	9	89	0131	59	01011001	Y	121	0171	79	01111001	y
26	0032	1A	00011010	SUB	58	0072	3A	00111010	:	90	0132	5A	01011010	Z	122	0172	7A	01111010	z
27	0033	1B	00011011	ESC	59	0073	3B	00111011	;	91	0133	5B	01011011	[	123	0173	7B	01111011	{
28	0034	1C	00011100	FS	60	0074	3C	00111100	<	92	0134	5C	01011100	\	124	0174	7C	01111100	
29	0035	1D	00011101	GS	61	0075	3D	00111101	=	93	0135	5D	01011101	]	125	0175	7D	01111101	}
30	0036	1E	00011110	RS	62	0076	3E	00111110	>	94	0136	5E	01011110	^	126	0176	7E	01111110	~
31	0037	1F	00011111	US	63	0077	3F	00111111	?	95	0137	5F	01011111	_	127	0177	7F	01111111	DEL

Exemple

char chr_a = 'a';	/* Déclaration d'une variable Char et initialisation de cette variable avec le caractère 'a' */
char chr_a_bis = 97;	/* Déclaration d'une variable Char et initialisation de cette variable avec le caractère numéro 97, soit le caractère 'a' */
char str[] = "arduino";	/* Déclaration d'une suite de caractère. La suite est stockée sous forme de tableau. La longueur du tableau est initialisée à la longueur du mot. */
char str2[8] = "arduino";	/* Idem ici, mais le tableau a déjà été initialisé à la bonne longueur du mot avant initialisation. */
char str3[15] = "arduino";	/* On peut également insérer une suite de caractères dans un tableau plus grand. */

unsigned char

La donnée « **unsigned char** » se comporte exactement comme un **char** auquel on ajoute les codes manquants (128 à 255) formant ainsi la **table ASCII étendue** (0 à 255):

DEC	OCT	HEX	BINARY	SYMBOL	DEC	OCT	HEX	BINARY	SYMBOL	DEC	OCT	HEX	BINARY	SYMBOL	DEC	OCT	HEX	BINARY	SYMBOL
128	0200	80	10000000	Ç	160	0240	A0	10100000	à	192	0300	C0	11000000	Ł	224	0340	E0	11100000	Ò
129	0201	81	10000001	ç	161	0241	A1	10100001	á	193	0301	C1	11000001	ł	225	0341	E1	11100001	ó
130	0202	82	10000010	é	162	0242	A2	10100010	ó	194	0302	C2	11000010	Ł	226	0342	E2	11100010	Ô
131	0203	83	10000011	â	163	0243	A3	10100011	û	195	0303	C3	11000011	ł	227	0343	E3	11100011	Õ
132	0204	84	10000100	ä	164	0244	A4	10100100	ñ	196	0304	C4	11000100	—	228	0344	E4	11100100	ö
133	0205	85	10000101	å	165	0245	A5	10100101	Ñ	197	0305	C5	11000101	Ł	229	0345	E5	11100101	Ö
134	0206	86	10000110	ä	166	0246	A6	10100110	ª	198	0306	C6	11000110	ä	230	0346	E6	11100110	µ
135	0207	87	10000111	ç	167	0247	A7	10100111	ë	199	0307	C7	11000111	Ä	231	0347	E7	11100111	þ
136	0210	88	10001000	ê	168	0250	A8	10101000	ç	200	0310	C8	11001000	Ł	232	0350	E8	11101000	ð
137	0211	89	10001001	ë	169	0251	A9	10101001	»	201	0311	C9	11001001	ł	233	0351	E9	11101001	ú
138	0212	8A	10001010	è	170	0252	AA	10101010	~	202	0312	CA	11001010	Ł	234	0352	EA	11101010	Û
139	0213	8B	10001011	í	171	0253	AB	10101011	½	203	0313	CB	11001011	ł	235	0353	EB	11101011	Ü
140	0214	8C	10001100	î	172	0254	AC	10101100	¾	204	0314	CC	11001100	Ł	236	0354	EC	11101100	Ý
141	0215	8D	10001101	ì	173	0255	AD	10101101	ı	205	0315	CD	11001101	—	237	0355	ED	11101101	Ÿ
142	0216	8E	10001110	Å	174	0256	AE	10101110	«	206	0316	CE	11001110	ł	238	0356	EE	11101110	˘
143	0217	8F	10001111	Ä	175	0257	AF	10101111	»	207	0317	CF	11001111	ı	239	0357	EF	11101111	˙
144	0220	90	10010000	É	176	0260	B0	10110000	ı	208	0320	D0	11010000	ð	240	0360	F0	11110000	-
145	0221	91	10010001	æ	177	0261	B1	10110001	ı	209	0321	D1	11010001	Ð	241	0361	F1	11110001	±
146	0222	92	10010010	Æ	178	0262	B2	10110010	ı	210	0322	D2	11010010	É	242	0362	F2	11110010	ˆ
147	0223	93	10010011	ò	179	0263	B3	10110011	ı	211	0323	D3	11010011	Ê	243	0363	F3	11110011	¾
148	0224	94	10010100	ó	180	0264	B4	10110100	ı	212	0324	D4	11010100	Ë	244	0364	F4	11110100	ŋ
149	0225	95	10010101	ô	181	0265	B5	10110101	À	213	0325	D5	11010101	ı	245	0365	F5	11110101	§
150	0226	96	10010110	ù	182	0266	B6	10110110	Á	214	0326	D6	11010110	ı	246	0366	F6	11110110	÷
151	0227	97	10010111	û	183	0267	B7	10110111	Â	215	0327	D7	11010111	ı	247	0367	F7	11110111	„
152	0230	98	10011000	ÿ	184	0270	B8	10111000	©	216	0330	D8	11011000	ı	248	0370	F8	11111000	*
153	0231	99	10011001	Û	185	0271	B9	10111001	ı	217	0331	D9	11011001	ı	249	0371	F9	11111001	˘
154	0232	9A	10011010	Ü	186	0272	BA	10111010	ı	218	0332	DA	11011010	ı	250	0372	FA	11111010	˙
155	0233	9B	10011011	ø	187	0273	BB	10111011	ı	219	0333	DB	11011011	ı	251	0373	FB	11111011	ı
156	0234	9C	10011100	ƒ	188	0274	BC	10111100	ı	220	0334	DC	11011100	ı	252	0374	FC	11111100	ı
157	0235	9D	10011101	ø	189	0275	BD	10111101	c	221	0335	DD	11011101	ı	253	0375	FD	11111101	ı
158	0236	9E	10011110	x	190	0276	BE	10111110	¥	222	0336	DE	11011110	ı	254	0376	FE	11111110	ı
159	0237	9F	10011111	f	191	0277	BF	10111111	ı	223	0337	DF	11011111	ı	255	0377	FF	11111111	nbsp

Exemple

```
unsigned char chr_c = 184; /* Déclaration d'une variable de type « unsigned char » est
                           initialisation avec le caractère © */
```

Qualifieurs des Variables et Portée

Les variables peuvent être **Qualifiées** par le biais des mots clés suivants

const	Préférez « const » à « #define » pour définir des constantes. <code>const [type] VARIABLENAME = [valeur] ;</code>
static	Variable dont la valeur est initialisée une seule fois au premier appel ou à sa déclaration.
volatile	Variable stockée en RAM plutôt que dans un registre de stockage. Ces variables sont utilisées en corrélation avec les "Interruptions" pour éviter les risques de corruption.

Outre ces Qualifieurs, les variables utilisées par langage Arduino possèdent toutes une **portée** dans le programme selon l'endroit où ces dernières sont déclarées. Les variables ne peuvent être utilisées que par des instructions qui se trouvent à l'intérieur de la fonction où ces variables ont été déclarées. On distingue 3 emplacements dans le code où une variable peut être déclarée :

- À l'intérieur d'une fonction ou d'un bloc de code. On les appelle les **variables locales**. Elles n'ont pour portée que le bloc ou la fonction où elles ont été déclarées.
- Dans les définitions des paramètres d'une fonction. Ces variables sont appelées ici des **paramètres**.
- En dehors de toute fonction ou bloc de programme. Ces variables sont des **variables globales**. Elles peuvent être utilisées dans l'ensemble du code.

Détaillons ici chacune de ces portées.

- Les variables à portée locales

Les variables déclarées à l'intérieur d'une fonction ou d'un bloc sont des variables locales. Ils ne peuvent être utilisés que par les instructions qui se trouvent à l'intérieur de cette fonction ou d'un bloc de code.

Exemple

```
void setup () {
}

void loop () {
  int i , j ;           // Déclaration de variables locales
  int z ;

  i = 0;                // Initialisation des variables
  j = 0;
  z = 12;
}

void fonctionQuelconque() {
  // Ici les variables i, j et z sont inconnues
}
```


- Les variables en tant que paramètre d'une fonction

Les variables peuvent être des paramètres d'une fonction. Prenons l'exemple d'une fonction d'addition de deux nombres entiers que l'on va appeler dans la boucle principale :

Exemple

```
void setup () {
}

/* Déclaration d'une fonction nommée Addition retournant un « int » et recevant 2 paramètres de
type « int » nbrA et nbrB */
int Addition(int nbrA, int nbrB) {

    int result = nbrA + nbrB ;    /* Initialisation d'une variable locale result qui contient le résultat
                                  du calcul nbrA + nbrB */

    return result ;
}

void loop () {

    int nombre = Addition(9,4); /* Déclaration d'une variable locale à la fonction loop() contenant le
                                résultat de la fonction Addition avec comme paramètres nbrA = 9 et
                                nbrB = 4 */

}
```

- Les variables à portée globales

Les variables globales sont définies en dehors de toutes les fonctions, généralement en haut du programme. Les variables globales conserveront leur valeur tout au long de la durée de vie de votre programme. Une variable globale peut être accédée par n'importe quelle fonction. Elle est disponible pour toute l'utilisation de votre programme après sa déclaration, peu importe la fonction où elle est appelée. Une variable globale peut également être une variable partagée qui servira à plusieurs fonctions.

Exemple

```
int nombre = 0;           // Initialisation d'une variable globale

void setup () {
}

/* Déclaration d'une fonction nommée Addition retournant un « int » et recevant 2 paramètres de
type « int » nbrA et nbrB */
int Addition(int nbrA, int nbrB) {

    int result = nbrA + nbrB ;    /* Initialisation d'une variable locale result qui contient le résultat
du calcul nbrA + nbrB */

    return result ;
}

void loop () {

    nombre = Addition(9,4);        /* On place le résultat de la fonction Addition avec comme
paramètres nbrA = 9 et nbrB = 4 dans la variable globale. Celle-ci
est maintenant égale à 13 */

    nombre = Addition(nombre,12);  /* On réadditionne le nombre avec 12. « nombre » vaut
maintenant 25 */

}
```

Utilitaires

PROGMEM	Variable stockée en mémoire Flash (programme) plutôt qu'en SRAM. Par exemple pour afficher des chaînes « string » assez longue, préférer ce type de variable afin de limiter l'utilisation SRAM. Dans ce cas, utiliser la macro F() comme : Serial.print(F("Write something on the Serial Monitor that is stored in FLASH"));
sizeof()	Retourne le nombre d'octets dans un type de variable, ou le nombre d'octets occupés par un tableau « array ». size_t taillevar = sizeof(variable)

Mot clé

void	Ce mot clé est utilisé uniquement dans les déclarations de fonction. La spécificité du « void » c'est que <u>la fonction associée ne retourne aucun résultat</u> lorsque celle-ci est appelée.
----------------------	---

Exemple

```
void loop ( ) {
    ...           //
    ...           // le code à exécuter ici
    ...           //
}
```

Partie 3 – Les Structures

Sketch.ino ou main.cpp (programme principal)

setup()	Fonction appelée une seule fois au démarrage du programme. Utilisez-la pour initialiser les variables, le mode d'utilisation des « pin », l'initialisation des bibliothèques, etc.
loop()	Fonction appelée <u>indéfiniment (en boucle)</u> juste après le setup(). Elle permet au programme d'agir (écouter, répondre). Utilisez-la pour contrôler activement la carte IoT.

Structures de Contrôle

if...else...	Structure « if...else... », « si » simple
<pre> if (condition) { ...; // traitement(s) si condition respectée } else { ...; // sinon traitement(s) condition non respectée } </pre>	
if...else if...else...	Structures « if...else if...else... », « si » imbriquées
<pre> if (condition 1) { ...; // traitement(s) si condition 1 respectée } else if (condition 2) { ...; // sinon traitement(s) si condition 2 respectée } else { ...; // sinon traitement(s) aucune condition respectée } </pre>	
do...while	Structure « do...while », « faire...tant-que » ; 1 à <i>n</i> boucles
<pre> do { ...; // traitement(s) tant que condition est respectée } while (condition) ; </pre>	
while	Structure « while », « tant-que » ; 0 à <i>n</i> boucles
<pre> while (condition) { ...; // traitement(s) tant que condition est respectée } </pre>	

Structures de Contrôle (suite)

for	Structure « for », « pour »
<pre>for (initialisation var; condition; incrément) { ... ; // traitement(s) pour chaque incrément de var }</pre>	
switch...case	Structure « switch...case », « selon...cas »
<pre>switch (var) { case valeur1: ...; // traitement(s) si var == valeur1 break; case valeur2: ...; // traitement(s) si var == valeur2 break; default: ...; // traitement(s) par défaut si var ne correspond à aucune valeur break; }</pre>	
continue	Retour en début d'une structure itérative avec passage à l'itération suivante
break	Provoque la sortie immédiate d'une structure
goto	Saut à une étiquette (<u>ne pas utiliser ce contrôle</u>)
return	Retour avec ou sans résultat d'une fonction
<pre>return ; // retour sans résultat return var; // retour avec la valeur de var</pre>	

Structure d'affectation particulière

?	Structure « si » condensée
<pre>variable = (condition ? valeur si condition respectée : valeur condition non respectée) ;</pre>	

Autres Eléments Structurant

#define	Directive de définition d'une donnée constante. Préférez l'utilisation du mot clé « const »
#include	Directive d'inclusion de bibliothèque externe permettant d'augmenter les fonctionnalités du programme sur une thématique spécifique.
/* */ (block comment)	Block de commentaire multilignes
// (single line comment)	Commentaire sur une seule ligne
; (semicolon)	Fin de chaque ligne de déclaration
{} (curly braces)	Début et Fin d'un bloc de plusieurs lignes de déclaration

Opérateurs Arithmétiques

% (remainder)	Reste de la division de donnée entière
* (multiplication)	Multiplication
+ (addition)	Addition
- (subtraction)	Soustraction
/ (division)	Division
= (assignment operator)	Affectation

Opérateurs de Comparaison

!= (not equal to)	Différent de
< (less than)	Inférieur à
<= (less than or equal to)	Inférieur ou Egal à
== (equal to)	Strictement égal à
> (greater than)	Supérieur à
>= (greater than or equal to)	Supérieur ou Egal à

Opérateurs Bit à Bit

& (bitwise and)	ET Binaire (bit à bit)
<< (bitshift left)	Glissement à gauche de 'x' bits ; $b = a \ll 3$
>> (bitshift right)	Glissement à droite de 'x' bits ; $b = a \gg 3$
^ (bitwise xor)	XOR (OU EXCLUSIF) Binaire (bit à bit)
 (bitwise or)	OU Binaire (bit à bit)
~ (bitwise not)	NOT Binaire (bit à bit)

Opérateurs Booléens

! (logical not)	Non Logique
&& (logical and)	ET Logique
 (logical or)	OU Logique

Opérateurs Composés

%= (compound remainder)	$x \%= \text{divisor};$ // equivalent to $x = x \% \text{divisor};$
&= (compound bitwise and)	$x \&= y;$ // equivalent to $x = x \& y;$
*= (compound multiplication)	$x *= y;$ // equivalent to $x = x * y;$
++ (increment)	$x++;$ // equivalent to $x = x + 1;$
+= (compound addition)	$x += y;$ // equivalent to $x = x + y;$
-- (decrement)	$x--;$ // equivalent to $x = x - 1;$
-= (compound subtraction)	$x -= y;$ // equivalent to $x = x - y;$
/= (compound division)	$x /= y;$ // equivalent to $x = x / y;$
^= (compound bitwise xor)	$x ^= y;$ // equivalent to $x = x ^ y;$
 = (compound bitwise or)	$x = y;$ // equivalent to $x = x y;$

Opérateurs d'accès aux Pointeurs

& (reference operator)	L'opérateur « esperluette & » est utilisé pour récupérer l'adresse mémoire d'une variable. Si 'x' est une variable, alors « &x » représente l'adresse de la variable 'x'.
* (dereference operator)	L'opérateur « astérisque * » est utilisé pour récupérer la valeur d'une variable représentée par son pointeur. Si 'p' est un pointeur, alors « *p » représente la valeur contenue dans l'adresse mémoire pointée par p.

Exemple

<pre>int *p; int i = 5; int result = 0; p = &i; result = *p;</pre>	<pre>// déclaration d'un pointer sur une donnée de type « int » // déclaration de la donnée de type « int » de valeur 5 /* maintenant le pointeur 'p' contient l'adresse mémoire de la donnée 'i' */ /* 'result' prend la valeur de la donnée pointée par 'p', il prend donc la valeur de 'i' soit 5 */</pre>
--	--

Les fichiers composants un projet Arduino

Avec l'IDE CLion associé au plugin PlatformIO, la composition d'un projet Arduino comprend, à minima, les 2 fichiers suivants :

- **main.cpp** – Le fichier principal du programme (équivalent au fichier « sketch.ino » de l'IDE historique Arduino) contenant obligatoirement les fonctions **setup()** et **loop()**
- **platformio.ini** – Le fichier de description de la plateforme matérielle et des diverses directives de compilation comme les bibliothèques externes à inclure, les moyens de communications à utiliser, etc. Ce fichier est spécifique à l'utilisation du plugin « PlatformIO ».

Le fichier « main.cpp »

```
#include <Arduino.h>           // inclusion de la bibliothèque contenant le langage arduino

void setup() {
    ...;                        // code exécuté une seule fois en début de programme
}

void loop() {
    ...;                        // code exécuté indéfiniment (en boucle)
}
```

Le fichier « platformio.ini »

```
; PlatformIO Project Configuration File
; -- Définition Environnement Matériel (ESP8266 – ESP12E)
[env:esp12e]

; Déclaration platform (famille), board (carte/module), framework (langage)
platform = espressif8266
board = esp12e
framework = arduino

; Déclaration du port utilisé pour la console série (+vitesse en 'bauds')
monitor_port = COM[3]1
monitor_speed = 115200

; Déclaration du port utilisé pour l'upload du module (+vitesse en 'bauds')
upload_port = COM[3]1
upload_speed = 115200
```

La zone grisée ci-dessus, n'est pas présente dans le fichier à sa création. Il est préférable d'ajouter ses lignes pour stipuler le port de communication et la vitesse de dialogue, entre notre ordinateur et le module IoT, pour les téléversements (upload) ou les logs (monitor).

¹: Remplacer le N° de COM utilisé (ici 3) par celui correspondant au port sur lequel l'IoT est connecté à votre ordinateur. Pour cela, il est possible de consulter le « **Gestionnaire de périphériques** » et relever le port sur lequel figure le libellé « CH340 » dans la zone « Ports 'COM et LPT' ».

L'arborescence typique d'un projet Arduino

 Project_Name	Dossier principal portant le nom du projet
-  .pio	<i>Dossier privé contenant le binaire « firmware.bin » du projet compilé</i>
-  data	Dossier contenant la structure et les éléments à envoyer dans le système de fichier (si implémenté).
-  include	Dossier contenant les fichiers d'entêtes « .h » génériques
-  lib	Dossier contenant les librairies personnelles sous forme de dossiers incluant chacun, à minima, un fichier d'en-tête « .h » et un fichier d'implémentation « .cpp »
-  src	Dossier contenant le fichier source du projet
 main.cpp	Fichier source du projet
-  test	Dossier contenant les éléments de tests (si implémenté).
 .gitignore	Fichier contenant le nom ou alias des éléments à ignorer par le gestionnaire de sources Git.
 platformio.ini	Fichier de description de l'environnement
 External Libraries	Dossier incluant toutes les librairies externes nécessaire au projet

Les Extensions de Fichiers « .cpp » et « .h »

Un fichier enregistré avec l'extension de fichier « .h » est un **fichier d'en-tête** utilisé dans les fichiers C/C++ pour inclure la déclaration des variables, des constantes et des fonctions (prototypes). Ceux-ci sont référencés par les fichiers d'implémentation C++ qui contiennent l'implémentation réelle de ces fonctions. Un fichier d'en-tête « .h » peut également inclure des informations supplémentaires telles que les définitions de macro. Ces fichiers d'en-tête sont référencés dans les fichiers C/C++ à l'aide de la directive `#include <monfichier.h>`.

Les fichiers avec l'extension de fichier « .cpp » sont des fichiers de code source pour les applications écrites en langage de programmation C++. Un seul projet C++ peut contenir plusieurs fichiers CPP en tant que code source de l'application. Un tel projet est constitué de différents types de fichiers, dont les fichiers « .cpp » qui sont appelés **fichiers d'implémentation** car ils contiennent toutes les définitions des méthodes déclarées dans le fichier d'en-tête « .h »